

Behavior Flows

This document traces the 8 core behaviors of the Architecture Model Standard, showing function signatures, return types, and sub-component involvement for each flow.

ID	Name	Trigger
BEH-INIT	Project Initialization	CLI: <code>architecture-model init <path></code>
BEH-VALIDATE	Model Validation	CLI: <code>architecture-model validate <model.yaml></code>
BEH-MANIFEST	Manifest Generation	CLI: <code>architecture-model manifest <path></code>
BEH-ENRICH	Auto-Enrichment	CLI: <code>architecture-model enrich <model.yaml></code>
BEH-EXTRACT	Model Extraction from Code	CLI/API
BEH-SLICE	Model Slicing	Programmatic API
BEH-DIFF	Model Diffing	Programmatic API
BEH-MERGE	Model Merging/Decomposition	Programmatic API

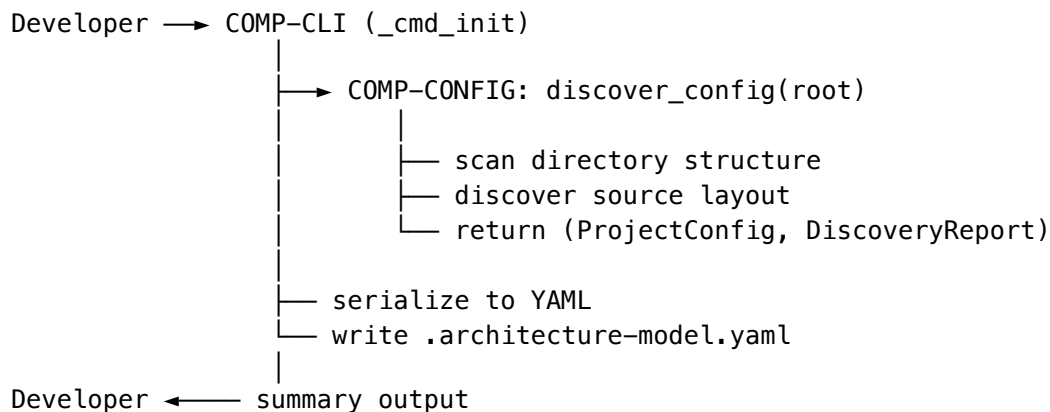
BEH-INIT: Project Initialization

Trigger: `architecture-model init <path>`

Component trace: COMP-CLI □ COMP-CONFIG

Steps

1. CLI dispatches to `_cmd_init(path)`
2. `discover_config(root: Path) -> tuple[ProjectConfig, DiscoveryReport]` (COMP-CONFIG)
 - Infers project name from directory name
 - Scans for source layout (src-layout, flat-layout, lib-layout)
 - Each subpackage becomes a Functional Block with files enumerated
 - Returns `(ProjectConfig, DiscoveryReport)` with layers, blocks, metrics
3. Serializes `ProjectConfig` to YAML
4. Writes `.architecture-model.yaml` to project root



Postconditions: `.architecture-model.yaml` exists with auto-discovered blocks. No source code modified.

BEH-VALIDATE: Model Validation

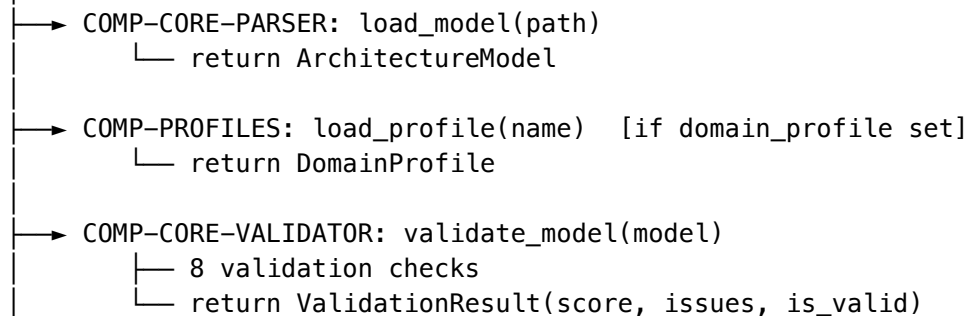
Trigger: `architecture-model validate <model.yaml> [--strict]`

Component trace: COMP-CLI □ COMP-CORE-PARSER □ COMP-CORE-VALIDATOR □ COMP-PROFILES

Steps

1. `load_model(path: Path) -> ArchitectureModel (COMP-CORE-PARSER)`
2. If `meta.domain_profile` is set: `load_profile(name: str) -> DomainProfile (COMP-PROFILES)`
3. `validate_model(model: ArchitectureModel) -> ValidationResult (COMP-CORE-VALIDATOR)`
 - ID uniqueness check
 - Referential integrity check
 - Orphan detection
 - Status consistency
 - Capability realization
 - Meta completeness
 - v1.1 semantics (data-model fields, state-machine reachability)
 - Regen readiness (constant/signature coverage)
 - If domain profile loaded: profile-specific validation rules
4. `ValidationResult` contains: `score: int (0-100)`, `issues: list[ValidationIssue]`, `is_valid: bool`
5. If `--strict`: warnings promoted to errors

Developer → COMP-CLI (validate)



Developer ← score + issues (exit 0 if valid, 1 if invalid)

BEH-MANIFEST: Manifest Generation

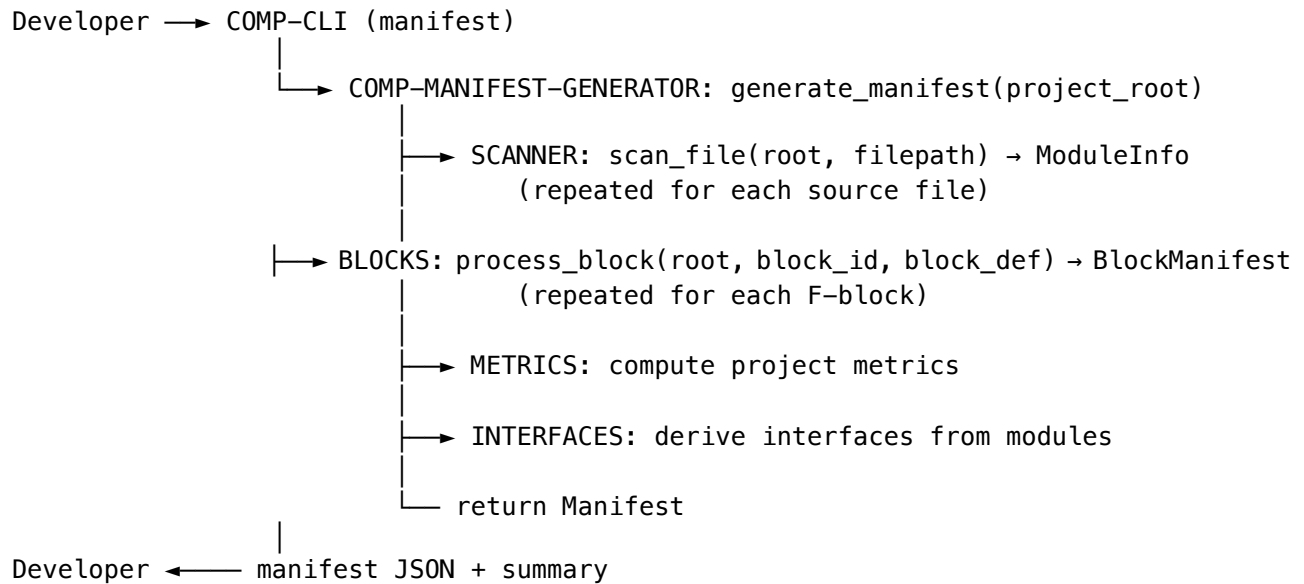
Trigger: `architecture-model manifest <path> [-o output]`

Component trace: COMP-CLI □ COMP-MANIFEST-GENERATOR □ {SCANNER, BLOCKS, METRICS, INTERFACES}

Steps

1. `generate_manifest(project_root: Path) -> Manifest (COMP-MANIFEST-GENERATOR)`

- Internally dispatches to sub-components:
 - a. **SCANNER**: `scan_file(root: Path, filepath: Path) -> ModuleInfo` — AST-scans each source file
 - b. **BLOCKS**: `process_block(root, block_id, block_def) -> BlockManifest` — processes each F-block
 - c. **METRICS**: computes project-level metrics (LOC, file count, complexity)
 - d. **INTERFACES**: derives interface inventory from scanned modules
- 2. Returns Manifest dataclass (use `.to_dict()` for JSON serialization)
- 3. Includes ScanReport with timing and counts



Postconditions: reality-manifest.json written. Contains AST-derived inventory: modules, blocks, metrics, interfaces.

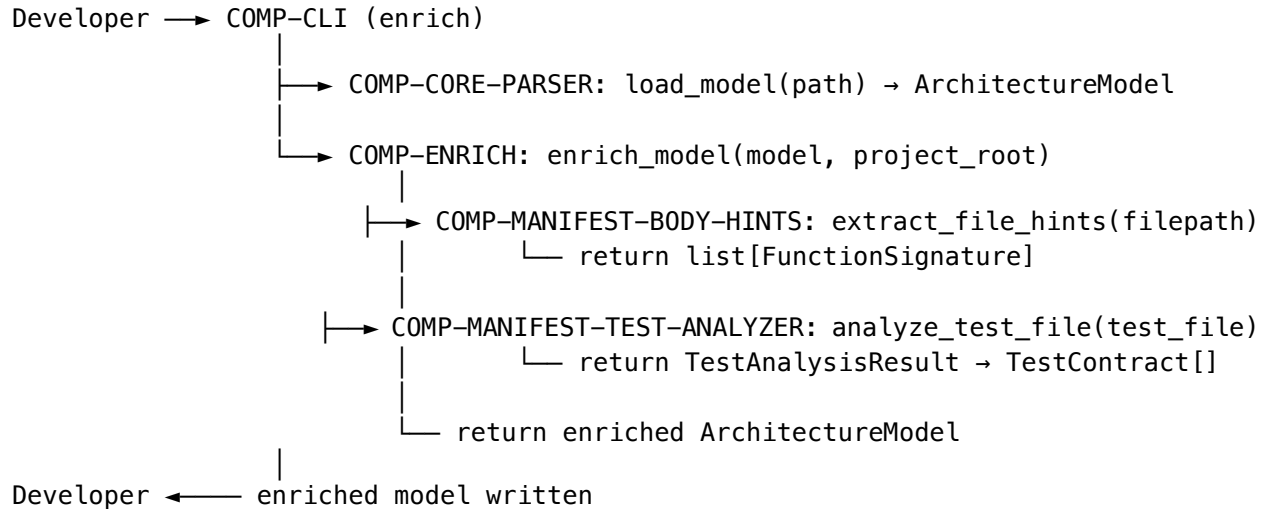
BEH-ENRICH: Auto-Enrichment

Trigger: `architecture-model enrich <model.yaml>`

Component trace: `COMP-CLI` □ `COMP-ENRICH` □ `{COMP-MANIFEST-BODY-HINTS, COMP-MANIFEST-TEST-ANALYZER}`

Steps

1. `load_model(path: Path) -> ArchitectureModel (COMP-CORE-PARSER)`
2. `enrich_model(model: ArchitectureModel, project_root: Path) -> ArchitectureModel (COMP-ENRICH)`
 - For each component in the model:
 - a. `extract_file_hints(filepath: Path) -> list[FunctionSignature] (COMP-MANIFEST-BODY-HINTS)`
 - Extracts body_hint, constants, class attributes via AST
 - b. `analyze_test_file(test_file: Path) -> TestAnalysisResult (COMP-MANIFEST-TEST-ANALYZER)`
 - Extracts test contracts with expected inputs/outputs
 - Attaches FunctionSignature and TestContract data to component entities
3. Returns enriched ArchitectureModel with AST-level detail



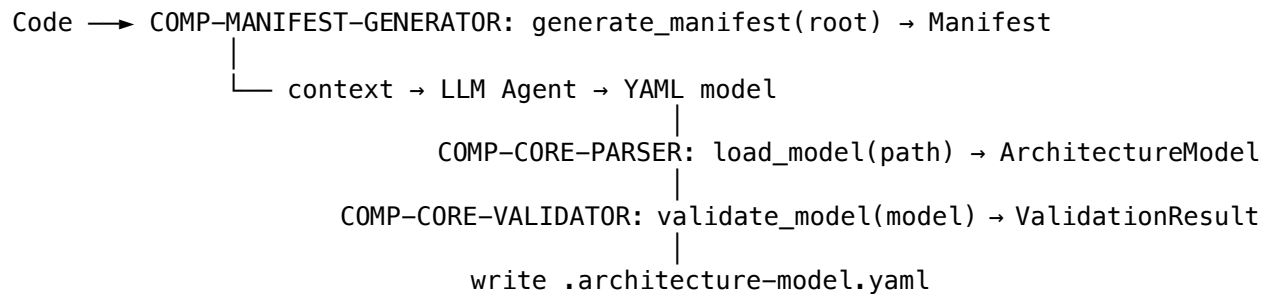
BEH-EXTRACT: Model Extraction from Code

Trigger: CLI or MCP architect_extract tool

Component trace: COMP-CLI □ COMP-MANIFEST-GENERATOR □ COMP-CORE-PARSER □ COMP-CORE-VALIDATOR

Steps

1. generate_manifest(project_root: Path) → Manifest — scan codebase for ground truth
2. LLM agent consumes manifest context and produces YAML model
3. load_model(path: Path) → ArchitectureModel — parse the generated model
4. validate_model(model: ArchitectureModel) → ValidationResult — verify structural correctness



BEH-SLICE: Model Slicing

Trigger: Programmatic API call

Component trace: COMP-CORE-SLICER

API

slice_by_fblock(model: ArchitectureModel, fblock_id: str) → ArchitectureModel
 slice_by_layer(model: ArchitectureModel, layer_id: str) → ArchitectureModel

Both return a new `ArchitectureModel` containing only entities and relationships relevant to the specified F-block or layer. Used by `format_fblock_context()` in the LLM integration layer to produce focused context windows.

BEH-DIFF: Model Diffing

Trigger: Programmatic API call

Component trace: COMP-CORE-DIFFER

API

`diff_models(old: ArchitectureModel, new: ArchitectureModel) -> ModelDiff`

Compares two model versions. `ModelDiff` contains added, removed, and modified entities/relationships. Used for change tracking and impact analysis.

BEH-MERGE: Model Merging and Decomposition

Trigger: Programmatic API call

Component trace: COMP-CORE-MERGER, COMP-CORE-DECOMPOSER

API

`merge_models(base: ArchitectureModel, overlay: ArchitectureModel) -> ArchitectureModel`
`decompose_model(model: ArchitectureModel) -> list[ArchitectureModel]`

- **Merge:** Combines two models, with overlay taking precedence on conflicts.
 - **Decompose:** Splits a model into per-subsystem sub-models for parallel regeneration.
-

BEH-PROFILE: Domain Profile Loading

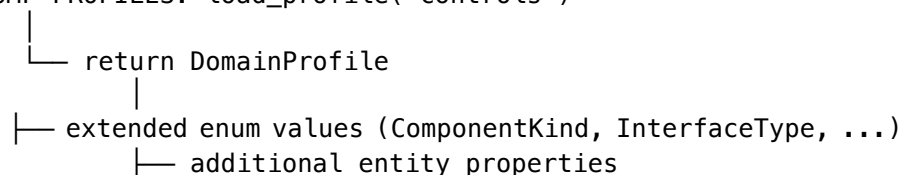
Trigger: During validation or parsing when `meta.domain_profile` is set

Component trace: COMP-PROFILES □ COMP-CORE-TYPES

Steps

1. `load_profile(name: str) -> DomainProfile (COMP-PROFILES)`
 - Loads profile definition (software, controls, mechanical, electrical)
 - Adds domain-specific enum values, properties, and validation rules
2. Open enum parsing: `ComponentKind.parse("sensor") (COMP-CORE-TYPES)`
 - Unknown values accepted as custom domain kinds rather than rejected

Parser/Validator → COMP-PROFILES: `load_profile("controls")`



└ conditional validation rules
|
COMP-CORE-TYPES: ComponentKind.parse("sensor") → ComponentKind